LANKETA – IV

Programazio Konkurrentearen Arazoak

Java bidezko analisia, simulazioak eta konponbide praktikoak

1T TALDEA: Julen | Iker | Ander U

Modulua: 0490 - Zerbitzu eta Prozesuen Programazioa (2. Maila)

Ikastetxea: Goierri Eskola | 1. Ebaluaketa

Aurkibidea: Landutako 5 Arazo Nagusiak

1T TALDEA
Julen | Iker | Ander U

1. **Interbloqueo o Deadlock**
   Bi hari edo gehiago baliabideak elkarri blokeatuta elkarren zain geratzen direnean (itxaronaldi zirkularra).

2. **Muerte por Inanición (Starvation)**
   Hari batek sekula ez duenean lortzen baliabidea, beste batzuek etengabe monopolizatzen dutelako.

3. **Condiciones de Carrera (Race Conditions)**
   Partekatutako datuetan eragiketa ez-atomikoak aldi berean egitean datu-galerak gertatzen direnean.

4. **Inconsistencia de Memoria**
   PUZaren cache lokalaren eraginez hari batek beste hariaren memoria-aldaketak ikusten ez dituenean.

5. **Condiciones Deslizadas (Slipped Conditions)**
   'Check-then-act' arrakala: baldintza egiaztatu eta ekintza burutu bitartean egoera aldatzen denean.

1. Interbloqueo o Deadlock (Blokeo Elkarrekikoa)

1T TALDEA
Julen | Iker | Ander U

Zer da Deadlock egoera bat?

• Definizioa: Bi hari edo gehiago modu mugagabean blokeatuta geratzen dira, bakoitzak besteak behar duen baliabidea askatu arte itxaroten duelako.
• Ondorioa: Prozesua edo zerbitzaria guztiz izoztuta geratzen da eta eskuz birabiarazi behar izaten da.
• Adibide erreala: Bi transakziok elkarren banku-kontuak blokeatzen dituztenean aldi berean diru-transferentzia bat egiteko.

Coffman-en 4 Baldintzak (Derrigorrezkoak)

1. Elkarrekiko Bazterketa (Mutual Exclusion):
Baliabidea ezin dute bi hariek aldi berean erabili.
2. Eutsi eta Itxaron (Hold and Wait):
Hari batek baliabide bat atxikitzen du bigarren baten zain dagoela.
3. Desjabetzerik Eza (No Preemption):
Ezin zaio hari bati bere baliabidea indarrez kendu; berak askatu behar du.
4. Itxaronaldi Zirkularra (Circular Wait):
Mendekotasun begizta itxia sortzen da (H1 -> B2 -> H2 -> B1).

1. Deadlock: IntelliJ Kodea, Exekuzioa eta Konponbidea

1T TALDEA
Julen | Iker | Ander U

DeadlockAdibidea.java - Kode Arriskutsua eta Konponbidea

```
// ARRISKUA: Hurrenkera gurutzatua (Itxaronaldi zirkularra)
Thread hari1 = new Thread(() -> {
    synchronized (baliabideA) {
        Thread.sleep(100);
        synchronized (baliabideB) { /* ... */ } // Zain geratzen da!
    }
});
Thread hari2 = new Thread(() -> {
    synchronized (baliabideB) {
        Thread.sleep(100);
        synchronized (baliabideA) { /* ... */ } // Zain geratzen da!
    }
});

// KONPONBIDEA: Hurrenkera BEREAN eskuratu blokeoak (A gero B)
// Bi hariek lehenik baliabideA eta gero baliabideB eskatuko dute beti.
```

IntelliJ Terminal - 1. Deadlock Exekuzioa

```
>>> 1. DEADLOCK (BLOKEO ELKARREKIKOA) - ARRISKUA <<<
[Hari-1] BaliabideA blokeatu du.
[Hari-2] BaliabideB blokeatu du.
[Hari-1] BaliabideB eskuratzen saiatzen... (zain)
[Hari-2] BaliabideA eskuratzen saiatzen... (zain)

[OHARRA / DEADLOCK DETEKTATUA!] Bi hariak blokeatuta geratu dira elkarren zain!
[Hari-1] egoera: BLOCKED (BaliabideB eskuratzeko zain)
[Hari-2] egoera: BLOCKED (BaliabideA eskuratzeko zain)

>>> 1. DEADLOCK - KONPONBIDEA (Hurrenkera Bera Erabiliz) <<<
[Hari-1 Zuzena] BaliabideA blokeatu du.
[Hari-1 Zuzena] BaliabideB ere blokeatu du eta lana amaitu du.
[Hari-2 Zuzena] BaliabideA blokeatu du.
[Hari-2 Zuzena] BaliabideB ere blokeatu du eta lana amaitu du.
[ONDO!] Bi hariak arazorik gabe amaitu dira Deadlock-ik gabe.
```

NOLA KONPONTZEN DA? (Resource Ordering / Blokeoen Hurrenkera Bera)

Hari guztiek blokeoak beti ordena BEREAN eskatzen badituzte (lehenik BaliabideA eta gero BaliabideB), ez da inoiz itxaronaldi zirkularrik sortuko eta Deadlock-a guztiz saihesten da!

2. Muerte por Inanición / Starvation (Baliabide-Gosetea)

1T TALDEA
Julen | Iker | Ander U

Zer da Inanizioa (Starvation)?

• Kontzeptua: Hari batek behar duen baliabidea edo PUZ denbora sekula ez lortzea, beste hari batzuek hura etengabe bereganatzen dutelako.
• Zergatia: Harien lehentasun desorekatuak (Thread.MAX_PRIORITY vs MIN_PRIORITY) edo blokeo bidegabeak (Unfair Locks).
• **Deadlock vs Inanizioa:**
- Deadlock: Hariak blokeatuta daude; inor ezin da aurrera atera.
- Inanizioa: Haria exekutatzeko prest dago (RUNNABLE), baina planifikatzaileak beste hariei ematen die lehentasuna beti.

Arriskuak eta Konponbide Bidea

• **Eragin Praktikoa:**
Atzeko planoko mantentze-atazak, log-idazleak edo bezero arrunten eskaerak guztiz geldituta geratzea sistema gainkargatuta dagoenean.
• **Konponbide Teknikoa: Justizia (Fairness):**
1. java.util.concurrent.locks.ReentrantLock(true) erabili, sarbidea FIFO (iritsi ahala zerbitzatu) ordenan emateko.
2. Lehentasun desorekatu artifizialak saihestea eta hari oldarkorrei 'Thread.sleep()' edo 'yield()' bidez arnasa ematea.

2. Inanizioa: IntelliJ Kodea, ReentrantLock eta Exekuzioa

1T TALDEA
Julen | Iker | Ander U

InanizioAdibidea.java - Lehentasunak vs ReentrantLock(true)

```
// ARRISKUA: Lehentasun desberdinak eta bidegabekeria
Thread hariAltua1 = new Thread(ataza);
hariAltua1.setPriority(Thread.MAX_PRIORITY); // Lehentasun gorena (10)

Thread hariBaxua = new Thread(ataza);
hariBaxua.setPriority(Thread.MIN_PRIORITY);  // Lehentasun baxuena (1)
// Hari baxuak ia ez du CPU edo blokeo-txandarik lortzen!

// KONPONBIDEA: ReentrantLock justizia bermearekin (Fairness)
final ReentrantLock fairLock = new ReentrantLock(true); // fair = true
fairLock.lock();
try {
    // Lan kritikoa egiten da FIFO (First-In, First-Out) ordenan
} finally {
    fairLock.unlock();
}
```

IntelliJ Terminal - 2. Inanizioa (Starvation) Exekuzioa

```
>>> 2. INANIZIOA (STARVATION) - ARRISKUA <<<
[Hari ALTUA 1] Bukatuta. Lortutako sarbideak: 7.982.843
[Hari ALTUA 2] Bukatuta. Lortutako sarbideak: 7.829.451
[Hari BAXUA (Gosez)] Bukatuta. Lortutako sarbideak: 6.786.502
OHARRA: Hari indartsuek etengabe monopolizatu dute monitorea!

>>> 2. INANIZIOA - KONPONBIDEA (Fair Lock bidez) <<<
java.util.concurrent.locks.ReentrantLock(true) erabiliz ('fairness' = true)
[Hari-3 (Max)]  Bidezko sarbideak: 11
[Hari-1 (Min)]  Bidezko sarbideak: 12
[Hari-2 (Norm)] Bidezko sarbideak: 12

[ONDO!] 'Fairness' aktibatuta, hari guztiek aukera orekatuak izan dituzte!
```

NOLA KONPONTZEN DA? (ReentrantLock bidezko Justizia / Fairness)

'new ReentrantLock(true)' erabilita, blokeoa itxaron-ilaran lehenago sartu den hariarentzat izango da (FIFO). Terminaleko irteeran ikus daitekeenez, hari guztiek sarbide kopuru orekatua lortzen dute (11, 12, 12)!

3. Condiciones de Carrera / Race Conditions (Lasterketa-Baldintza)

1T TALDEA
Julen | Iker | Ander U

Zer da Lasterketa-Baldintza bat?

• Definizioa: Hari anitz aldi berean partekatutako aldagai berera atzitzen direnean eta gutxienez batek idazten duenean, emaitza harien exekuzio-ordenaren mende geratzen da (ez-deterministikoa).
• Zergatik gertatzen da? Eragiketa asko ez dira atomikoak. Adibidez, kontagailua++ eragiketak 3 instrukzio ditu:

1. READ: Memoriako balioa irakurri erregistrora.
2. MODIFY: Balioari 1 gehitu erregistroan.
3. WRITE: Balio berria memorian gorde.

Bi harik aldi berean 1. pausoa egiten badute, balio bera irakurriko dute eta batak bestearen gehiketa gainidatziko du!

Konponbideak Javan

1. 'synchronized' Metodo edo Blokeak:
Objektuaren monitorea (Lock) eskuratzen du. Une bakoitzean haria bakar bat sar daiteke sekzio kritikoan, beste guztiei sarrera itxiz amaitu arte.

2. java.util.concurrent.atomic.AtomicInteger:
Blokeorik gabeko metodoa da. PUZ hardware mailako CAS (Compare-And-Swap) instrukzioak erabiltzen ditu; 'synchronized' baino askoz arinagoa eta eraginkorragoa da kontagailuetarako.

3. Race Conditions: IntelliJ Kodea eta Exekuzio Emaitza

1T TALDEA
Julen | Iker | Ander U

LasterketaBaldintzaAdibidea.java - kontagailua++ vs synchronized / Atomic

```java
// ARRISKUA: kontagailua++ ez da atomikoa (Read-Modify-Write)
private static int kontagailuNormala = 0;
// Bi harik 10.000 aldiz: kontagailuNormala++;
// Biek balio bera irakurtzen badute, batak bestea gainidazten du!

// 1. KONPONBIDEA: synchronized metodoa
private static synchronized void inkrementatuSinkronizatua() {
    kontagailuSinkronizatua++;
}

// 2. KONPONBIDEA: AtomicInteger (Hardware CAS bidez)
private static final AtomicInteger kontagailuAtomikoa = new AtomicInteger(0);
kontagailuAtomikoa.incrementAndGet(); // Atomikoa eta azkarra
```

IntelliJ Terminal - 3. Lasterketa-Baldintza (Race Condition) Exekuzioa

```
>>> 3. LASTERKETA-BALDINTZA (RACE CONDITION) - ARRISKUA <<<
Bi harik partekatutako kontagailua bakoitzak 10.000 aldiz gehituko dute.
Espero den emaitza teorikoa: 20.000
Lortutako azken balioa (Sinkronizatu gabe): 17.524
[ARAZOA!] Lasterketa-baldintza gertatu da! Galdu diren gehiketak: 2.476

>>> 3. LASTERKETA-BALDINTZA - KONPONBIDEA <<<
1. Emaitza 'synchronized' metodoarekin: 20000 / 20.000
2. Emaitza 'AtomicInteger'-ekin:        20000 / 20.000

[ONDO!] Bi konponbideek emaitza zehatza eta sendoa (thread-safe) bermatzen dute!
```

EMAITZEN ANALISIA: GALDU DIREN 2.476 GEHIKETAK

Sinkronizazio gabe: 20.000 espero ziren arren, 17.524 lortu dira (gainidazketak!). 'synchronized' edo 'AtomicInteger' erabilita berriz, emaitza beti %100 zehatza da: 20.000 / 20.000.

4. Inconsistencia de Memoria (Memoriaren Sendotasun-Eza)

1T TALDEA
Julen | Iker | Ander U

Zergatik gertatzen da Ikusgarritasun Arazoa?

• PUZ Modernoen Arkitektura: Prozesagailu bakoitzak bere cache memoria azkarrak ditu (L1, L2 eta erregistroak).
• Eraginkortasun-arazoa: Haria batek partekatutako aldagai bat irakurtzen duenean, bere cachean kopiatzen du azkarrago atzitzeko.
• Ondorioa: Beste hari batek aldagaia aldatu arren RAM nagusian, lehen haria bere cache lokal zaharretik irakurtzen jarraitzen du, eta ez da konturatzen balioa aldatu dela!
• JIT Optimizations: Konpilatzaileak 'while(!gelditu)' kodea 'while(true)' gisa optimiza dezake, bukle infinituan geratuz.

Konponbidea: 'volatile' eta Happens-Before

Zer bermatzen du 'volatile' hitz-gakoak?

1. Zuzeneko Idazketa: Aldagaiaren edozein aldaketa berehala idazten da RAM memoria nagusira, cacheetan gorde gabe.
2. Zuzeneko Irakurketa: Hari guztiek beti RAM memoria nagusitik irakurtzen dute uneko balio erreala.
3. 'Happens-Before' Bermeak: Java Memoria Ereduak (JMM) agintzen du idazketa ondorengo irakurketa ororen aurretik gertatuko dela.

GARRANTZITSUA: 'volatile'-k ikusgarritasuna bermatzen du, baina EZ atomikotasuna (adibidez, kontagailua++ bezalakoetan ez da nahikoa).

4. Memoria Inkontsistentzia: IntelliJ Kodea eta Exekuzioa

1T TALDEA
Julen | Iker | Ander U

MemoriaInkontsistentziaAdibidea.java - CPU Cache vs volatile

```
// ARRISKUA: Aldagai arrunta core bakoitzeko cachean gorde daiteke
private static boolean gelditu = false; // ez-volatile
// Langile-haria:
while (!gelditu) { /* PUZ cachetik irakurtzen du etengabe! */ }
// Hari nagusiak 'gelditu = true' jarri arren, langileak agian ez du IKUSIKO!

// KONPONBIDEA: volatile hitz-gakoa
private static volatile boolean banderaVolatile = false;
// 1. Idazketak zuzenean RAM memorian gordetzen dira berehala.
// 2. Irakurketak zuzenean RAM memoriatik egiten dira (Happens-Before).
```

IntelliJ Terminal - 4. Memoria Inkontsistentzia Exekuzioa

```
>>> 4. MEMORIAREN SENDOTASUN-EZA - KONPONBIDEA ('volatile') <<<
Aldagaia 'volatile' gisa deklaratzean:
private static volatile boolean banderaVolatile = false;

[Langile-Haria] Martxan... 'banderaVolatile = true' izateko zain.
[Hari Nagusia] 'banderaVolatile = true' ezartzen ari da...
[Langile-Haria] Banderaren aldaketa IKUSI DUT! Gelditzen naiz. (Iterazioak: 797.125.150)

[ONDO!] Langile-haria garaiz gelditu da, 'volatile'-k RAM bidezko ikusgarritasuna bermatu duelako
```

KODE ETA TERMINAL ANALISIA: 'volatile' EFEKTUA

Langile-haria bukle estu batean dabil 'banderaVolatile' aldagaia irakurtzen. Hari nagusiak balioa aldatu bezain laster, langileak berehala detektatzen du RAM-etik eta segurtasunez gelditzen da!

5. Condiciones Deslizadas / Slipped Conditions (Lerratutako Baldintzak)

1T TALDEA
Julen | Iker | Ander U

Zer da 'Check-then-Act' Arrakala?

• Definizioa: Lasterketa-baldintza mota espezifiko bat da; hari batek baldintza bat egiaztatzen du (Check), baina ekintza burutu aurretik (Act), beste hari batek baldintza hori baliogabetzen du.
• Kausa: Egiaztapena eta ekintza sinkronizazio bloke berean ez egotea (bi urrats bananduak izatea).
• **Banku-Kontuaren Adibidea:**
1. Kontuan 100 € daude.
2. Hari-1ek egiaztatzen du: '100 € badaude, banoa ateratzera...'
3. Bitartean Hari-2k ere 100 € daudela ikusi eta atera egiten ditu (Saldoa = 0 €).
4. Hari-1 bueltatu eta dirua ateratzen du: Saldoa = -100 € geratzen da!

Konponbide Atomikoa

Nola Saihestu Baldintza Labaintzea?

Egiaztapena (Check) eta Diru-Kentzea (Act) eragiketa bakar eta atomiko batean bateratu behar dira metodo edo bloke 'synchronized' baten barruan:

```
public synchronized boolean ateraDirua(int kantitatea) {
   if (saldoa >= kantitatea) {
      saldoa -= kantitatea;
      return true;
   }
   return false;
}
```

Horrela, Hari-1ek saldoa egiaztatzen duen unetik kentzen duen unera arte, ezin da Hari-2 sartu eta saldo negatiboa sortzea ezinezkoa da!

5. Lerratutako Baldintzak: IntelliJ Kodea eta Exekuzioa

1T TALDEA
Julen | Iker | Ander U

LerratutakoBaldintzakAdibidea.java - Check-then-Act arrakala vs Metodo Atomikoa

```java
// ARRISKUA: Check-then-Act bereizita (Baldintza lerratu egiten da)
if (kontua.getSaldoa() >= 100) { // 1. EGIAZTAPENA (Check)
    Thread.sleep(50);             // Testuinguru aldaketa (Hari 2 sartu da!)
    kontua.kenduDirua(100);        // 2. EKINTZA (Act) -> Saldo negatiboa!
}

// KONPONBIDEA: Egiaztapena eta ekintza bloke ATOMIKO bakarrean
public synchronized boolean ateraDiruaAtomikoki(String haria, int kantitatea) {
    if (saldoa >= kantitatea) {
        saldoa -= kantitatea;
        return true;
    }
    return false; // Saldo nahikorik ez badago, ezeztatu
}
```

IntelliJ Terminal - 5. Lerratutako Baldintzak Exekuzioa

```
>>> 5. LERRATUTAKO BALDINTZAK (SLIPPED CONDITIONS) - ARRISKUA <<<
Hasierako saldoa: 100€. Bi bezerok 100€ atera nahi dituzte.
[Hari-2] Egiaztapena: 100€ badira kontuan. Ateratzera noa...
[Hari-1] Egiaztapena: 100€ badira kontuan. Ateratzera noa...
[Hari-2] 100€ atera ditut! Uneko saldoa: -100€
[Hari-1] 100€ atera ditut! Uneko saldoa: -100€
AZKEN SALDOA: -100€ (ARAZOA! Saldo negatiboa sortu da!)

>>> 5. LERRATUTAKO BALDINTZAK - KONPONBIDEA <<<
[Hari-2] Saldo nahikoa dago (100€). Ateratzen: 100€ -> Saldo berria: 0€
[Hari-1] EZEZTATUA! Saldo gutxiegi (0€). Ezin dira 100€ atera.
AZKEN SALDOA (Konponduta): 0€
[ONDO!] Haria batek bakarrik atera du, bigarrenari ezezkoa eman zaio.
```

TERMINALEKO EMAITZA: SALDO NEGATIBOARI IRTENBIDEA

Kode arriskutsuan bi hariek atera dute dirua eta saldoa -100€ geratu da. Metodo atomiko sinkronizatuarekin, Hari-2k 100€ atera ondoren saldoa 0€ geratu da, eta Hari-1i ezezkoa eman zaio modu seguruan!

Laburpen Konparatiboa: 5 Arazoak Begiratu Batean

1T TALDEA
Julen | Iker | Ander U

Arazoa	Kausa Nagusia	Sintoma / Arriskua	Java Konponbidea
1. Deadlock	Itxaronaldi zirkularra baliabideetan	Programa betiko izoztuta geratzea	Baliabideak ordena berean eskuratu
2. Inanizioa	Baliabide-monopolioa / Bidegabekeria	Hari ahulak sekula ez exekutatzea	ReentrantLock(true) (Fairness)
3. Race Conditions	Eragiketa ez-atomikoak partekatuan	Datu-galerak eta emaitza okerrak	synchronized edo AtomicInteger
4. Memoria Inkontsist.	CPU Cacheak RAM-ekin sinkronizatu gabe	Hari batek ez ditu aldaketak ikusten	volatile (Happens-Before erlazioa)
5. Slipped Conditions	Check-then-act arrakala banandua	Egoera baliogabeak (saldo negatiboa)	Check eta Act 'synchronized' bakarrean

Ondorioak eta Galderak

1T TALDEA
Julen | Iker | Ander U

Ondorio Nagusiak

1. Programazio Konkurrentearen Konplexutasuna:
Hari anitzeko inguruneetan ohiko akatsak ez-deterministikoak dira: baliteke milioika aldiz ondo funtzionatzea eta zerbitzari produkzioan bezero asko daudenean huts egitea.

2. 'Thread Safety' Bermatzea Garatzailearen Ardura Da:
Konpilatzaileak ez du lasterketa-baldintzarik edo blokeorik debekatzen. Garatzaileak sinkronizazio-mekanismo egokiak (synchronized, volatile, atomic, Locks) propio erabili behar ditu.

3. Errendimendua vs Segurtasuna:
Sinkronizazio gehiegi jartzeak paralelismoa eta abiadura murrizten ditu (bottleneck); gutxiegi jartzeak datuak usteltzen ditu. Diseinu orekatua ezinbestekoa da.

Eskerrik asko zuen arretagatik! Galderarik?